

UNIVERSIDADE FEDERAL DE SERGIPE
CIDADE UNIVERSITÁRIA PROFESSOR JOSÉ ALOÍSIO DE CAMPOS

FRANCISCO PASSOS DOS SANTOS ALVES

IMPLEMENTAÇÃO DA SÉRIE DE TAYLOR USANDO OS FUNDAMENTOS DA
LÓGICA FUNCIONAL EM HASKELL

SÃO CRISTÓVÃO
2026

Sumário

1	Introdução	2
2	Séries de potência	3
2.1	Séries de Taylor	3
2.2	Função seno definida pela série de Taylor	3
2.3	Função cosseno definida pela série de Taylor	3
3	Desenvolvimento	4
3.1	Fundamentos funcionais na modelagem computacional	4
3.2	Representação de séries por meio de listas	5
3.3	Implementação e formalização das séries de Taylor	6
3.3.1	Série do seno	6
3.3.2	Série do cosseno	7
3.3.3	Observações sobre a implementação	7
4	Resultados	8
5	Conclusão	9
6	Referências	10

Resumo

A série de Taylor é uma ferramenta matemática fundamental na computação, permitindo aproximar funções contínuas e viabilizar o cálculo de valores de funções como seno e cosseno para diferentes entradas.

Neste trabalho, utiliza-se a linguagem Haskell, explorando conceitos do paradigma de programação funcional, com o objetivo de implementar essas aproximações e analisar seu comportamento tanto do ponto de vista matemático quanto computacional.

1 Introdução

O paradigma de programação funcional é uma abordagem fundamentada no cálculo lambda e baseada na avaliação de funções matemáticas, sendo caracterizado pela ausência de estados mutáveis e pelo uso de funções como elementos fundamentais de construção. O cálculo lambda consiste em um modelo formal de computação baseado na definição, aplicação e abstração de funções, no qual expressões são construídas por meio da aplicação de funções a argumentos e da substituição de variáveis. Diferentemente de outros paradigmas, como o imperativo e o orientado a objetos, a programação funcional enfatiza a composição de funções e o uso de recursão na resolução de problemas [1].

No contexto da computação científica, diversas funções matemáticas não podem ser calculadas de forma exata por meio de operações elementares, sendo necessário recorrer a métodos de aproximação. Entre esses métodos, destaca-se a série de Taylor, que permite representar funções como somas de termos derivados de suas derivadas em um ponto específico [3].

Do ponto de vista matemático, essa representação é exata dentro do seu domínio de convergência. No entanto, em contextos computacionais, torna-se necessário truncar a série, resultando em aproximações numéricas cujo grau de precisão depende da quantidade de termos considerados. Essa característica torna esse método especialmente relevante em computação científica [4].

Neste trabalho, será utilizada a linguagem Haskell para implementar essas aproximações, explorando conceitos do paradigma funcional, como recursão, imutabilidade e avaliação preguiçosa (lazy evaluation). O objetivo é analisar tanto o comportamento matemático das séries quanto sua aplicação prática em um contexto computacional.

2 Séries de potência

2.1 Séries de Taylor

Seja f derivável até a ordem n em um intervalo I e seja $x_0 \in I$. O polinômio

$$P_n(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

ou

$$P_n(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k$$

denomina-se polinômio de Taylor de ordem n de f em torno de x_0 [4].

2.2 Função seno definida pela série de Taylor

A função seno pode ser representada por uma série infinita de potências, conhecida como série de Taylor. Essa representação permite aproximar o valor de $\sin(x)$ por meio de um somatório de termos polinomiais. [4]

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots$$

ou

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} \quad (1)$$

Nessa expressão, observa-se que apenas potências ímpares de x são utilizadas, e os sinais alternam entre positivo e negativo. Além disso, essa série é convergente, ou seja, à medida que mais termos são adicionados, o valor obtido se aproxima do valor real da função. [5] Formalmente, a convergência é expressa por meio do limite da sequência de somas parciais, que permite determinar precisamente o valor para o qual a série converge:

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \frac{(-1)^k}{(2k+1)!} x^{2k+1} = \sin(x) \quad (2)$$

Na prática computacional, essa série é truncada em um número finito de termos para obter uma aproximação da função.

2.3 Função cosseno definida pela série de Taylor

De forma semelhante, a função cosseno também pode ser expressa como uma série de Taylor. Essa representação é composta por termos envolvendo potências pares de x . [4]

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \cdots$$

ou

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n} \quad (3)$$

Assim como no caso do seno, os sinais dos termos alternam, e a série é infinita. Essa série também é convergente, garantindo que a soma de uma quantidade finita de termos seja suficiente para obter boas aproximações. [5] De forma análoga, o limite da sequência de somas parciais determina o valor de convergência da série:

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \frac{(-1)^k}{(2k)!} x^{2k} = \cos(x) \quad (4)$$

Para aplicações computacionais, utiliza-se apenas uma quantidade finita de termos, resultando em uma aproximação da função.

3 Desenvolvimento

Nesta seção, serão apresentados os fundamentos teóricos e computacionais utilizados na construção do programa, bem como a forma como esses conceitos são aplicados na modelagem e implementação das séries de Taylor. Inicialmente, serão discutidos os princípios da programação funcional que sustentam a abordagem adotada, com destaque para o uso de funções como cidadãos de primeira classe, recursão e avaliação preguiçosa.

Em seguida, será abordada a representação de séries matemáticas por meio de listas, explorando a correspondência entre sequências definidas formalmente e sua implementação em Haskell. Essa representação permite a construção de estruturas potencialmente infinitas, cujos elementos são gerados sob demanda.

Posteriormente, será apresentada a implementação das séries de Taylor para as funções seno e cosseno, integrando tanto a descrição computacional quanto a formalização matemática, evidenciando como a definição matemática pode ser traduzida diretamente para o código por meio de compreensões de listas e funções de alta ordem.

3.1 Fundamentos funcionais na modelagem computacional

No paradigma de programação funcional, os programas são construídos a partir da composição de funções matemáticas, evitando o uso de estados mutáveis e efeitos colaterais. Essa abordagem permite que a lógica computacional seja descrita de forma declarativa, aproximando-se diretamente da notação matemática utilizada na formulação de problemas. [1]

Uma característica fundamental desse paradigma é o tratamento de funções como cidadãos de primeira classe (*first-class citizens*). Isso significa que funções podem ser atribuídas a variáveis, passadas como argumentos e retornadas como resultados de outras funções. Essa propriedade é essencial para a construção de abstrações mais expressivas e reutilizáveis.

Para ilustrar esse conceito, considere uma função que aplica uma operação matemática duas vezes consecutivamente. Em uma abordagem imperativa, seria necessário escrever instruções explícitas de repetição. Em programação funcional, a própria função pode ser passada como argumento:

```
-- Aplicar uma função duas vezes
aplicarDuasVezes :: (a -> a) -> a -> a
aplicarDuasVezes f x = f (f x)

-- Exemplos de uso:
```

```
-- aplicarDuasVezes (+1) 5 resulta em 7
-- aplicarDuasVezes (*2) 3 resulta em 12
```

Nesse exemplo, a função `aplicarDuasVezes` recebe como argumento outra função `f`, demonstrando como funções são tratadas como valores ordinários na linguagem.

Outro conceito importante é a recursão, que substitui estruturas iterativas tradicionais, como laços *for* e *while*. Em vez de repetir instruções por meio de variáveis de controle mutáveis, funções recursivas definem seus resultados em termos de si mesmas.

Um exemplo clássico é a função fatorial, definida como $fatorial : \mathbb{N} \rightarrow \mathbb{N}$, tal que:

$$fatorial(n) = \begin{cases} 1, & \text{se } n = 0 \\ n \cdot fatorial(n - 1), & \text{se } n > 0 \end{cases} \quad (5)$$

Em Haskell, essa definição pode ser implementada de forma direta:

```
fatorial :: Natural -> Natural
fatorial 0 = 1
fatorial n = n * fatorial(n - 1)
```

Além disso, a linguagem Haskell utiliza avaliação preguiçosa (*lazy evaluation*), na qual expressões são avaliadas apenas quando seus valores são necessários. Esse comportamento permite trabalhar com estruturas potencialmente infinitas, como listas que representam séries matemáticas.

Esses conceitos tornam a programação funcional particularmente adequada para a modelagem de séries de Taylor, uma vez que a própria definição matemática da série pode ser traduzida quase diretamente para o código, utilizando listas, funções de alta ordem (*higher-order functions*) e recursão. [1]

3.2 Representação de séries por meio de listas

Na linguagem Haskell, séries matemáticas podem ser representadas diretamente por meio de listas, explorando o conceito de avaliação preguiçosa, já discutido anteriormente. Esse mecanismo permite a definição de listas potencialmente infinitas, nas quais os elementos são computados sob demanda, somente quando necessários. [2]

Do ponto de vista matemático, uma lista pode ser interpretada como uma sequência indexada pelos números naturais, da forma:

$$(a_k)_{k \in \mathbb{N}}$$

onde cada termo a_k é gerado a partir de uma regra de formação. No caso das séries de Taylor, essa regra corresponde ao termo geral da série. [6]

Em Haskell, essa mesma ideia é expressa por meio de compreensão de listas:

```
[ a_k | k <- [0..] ]
```

Nessa construção, a expressão `k <- [0..]` define uma sequência infinita de índices naturais, enquanto a_k representa a regra de formação de cada termo da lista. [2]

Para ilustrar esse conceito de forma concreta, considere alguns exemplos simples de listas geradas por compreensão:

```
-- Números naturais a partir de zero
naturais = [k | k <- [0..]]
-- Resultado: [0, 1, 2, 3, 4, 5, ...]

-- Quadrados dos números naturais
quadrados = [k^2 | k <- [0..]]
-- Resultado: [0, 1, 4, 9, 16, 25, ...]

-- Números pares
pares = [2*k | k <- [0..]]
-- Resultado: [0, 2, 4, 6, 8, 10, ...]
```

Embora essas listas representem sequências infinitas, a avaliação preguiçosa garante que apenas os elementos necessários são computados. Ao utilizar a função `take`, é possível selecionar uma quantidade finita de termos:

```
-- Pega apenas os 5 primeiros quadrados
take 5 quadrados
-- Resultado: [0, 1, 4, 9, 16]

-- Pega os 3 primeiros pares
take 3 pares
-- Resultado: [0, 2, 4]
```

Dessa forma, estruturas potencialmente infinitas são avaliadas parcialmente, permitindo trabalhar com séries matemáticas sem a necessidade de armazenar todos os termos em memória.

No contexto deste trabalho, cada série de Taylor é modelada como uma lista de termos, em que cada elemento corresponde a um termo da expansão da função. Essa representação estabelece uma correspondência direta entre a definição matemática da série e sua implementação computacional, como será demonstrado na próxima seção.

Além disso, a utilização de listas permite o emprego de funções de alta ordem, como `take` e `sum`, que possibilitam selecionar uma quantidade finita de termos e realizar a soma desses elementos de forma declarativa. Dessa maneira, uma estrutura infinita pode ser utilizada para produzir resultados finitos, sem a necessidade de controle explícito de iteração.

3.3 Implementação e formalização das séries de Taylor

Após compreender os fundamentos da representação de séries por meio de listas, é possível implementar diretamente as séries de Taylor para as funções seno e cosseno em Haskell.

3.3.1 Série do seno

A função seno aproximada é definida como $\text{senT} : \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$, representando a soma parcial da série de Taylor associada à função seno:

$$\text{senT}(x, n) = \sum_{k=0}^{n-1} \frac{(-1)^k}{(2k+1)!} (x^{2k+1}) \quad (6)$$

Nessa definição, $x \in \mathbb{R}$ representa o valor de entrada, enquanto $n \in \mathbb{N}$ corresponde à quantidade de termos considerados na aproximação.

Em Haskell, essa definição é implementada de forma direta:

```
-- Série de Taylor do seno
senT :: Double -> Natural -> Double
senT x n = sum (take (fromIntegral n) [ ((-1) ^ k) * (x ^ (2*k + 1)) /
                                         fromIntegral(fatorial(fromIntegral(2*k + 1)))
                                         | k <- [0..]])
```

Observa-se que a implementação preserva a estrutura matemática da série. A lista infinita gerada pela compreensão de listas:

```
[ ((-1) ^ k) * (x ^ (2*k + 1)) /
  fromIntegral(fatorial(fromIntegral(2*k + 1)))
| k <- [0..] ]
```

representa os termos da série de Taylor do seno. A função `take n` seleciona apenas os primeiros n termos dessa lista infinita, e `sum` calcula a soma desses termos, resultando na aproximação desejada.

A função `fromIntegral` é responsável por converter valores inteiros em números de ponto flutuante, permitindo que as divisões sejam realizadas corretamente no contexto das operações numéricas. [2]

3.3.2 Série do cosseno

De forma análoga, a função cosseno aproximada é definida como $\cos T : \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$:

$$\cos T(x, n) = \sum_{k=0}^{n-1} \frac{(-1)^k}{(2k)!} (x^{2k}) \quad (7)$$

A implementação em Haskell segue o mesmo padrão:

```
-- Série de Taylor do cosseno
cosT :: Double -> Natural -> Double
cosT x n = sum (take (fromIntegral n) [ ((-1) ^ k) * (x ^ (2*k)) /
                                         fromIntegral(fatorial(fromIntegral(2*k)))
                                         | k <- [0..]])
```

Assim como na série do seno, a implementação em Haskell reflete diretamente a definição matemática. A lista infinita representa todos os termos potenciais da série, enquanto `take` e `sum` realizam o truncamento e a soma, respectivamente, produzindo uma aproximação numérica da função cosseno.

3.3.3 Observações sobre a implementação

Ambas as funções demonstram a expressividade da linguagem Haskell na modelagem de funções matemáticas. A correspondência entre a notação matemática e o código é quase direta, minimizando a distância entre a formulação teórica e a realização computacional.

Um detalhe importante é o uso do módulo `Numeric.Natural`, que define o tipo `Natural` para representar os números naturais $\mathbb{N}$ de forma matematicamente precisa. Ao

utilizar esse módulo, garante-se que o parâmetro n (quantidade de termos) pertence verdadeiramente ao conjunto dos naturais, evitando valores negativos em tempo de compilação. Isso reforça a ligação entre a formulação matemática e a implementação computacional, uma vez que a própria linguagem valida a correção do domínio da função.

Vale ressaltar que a função `fatorial`, embora correta, possui complexidade temporal linear em relação ao seu argumento. Para valores de n muito grandes, essa implementação pode se tornar ineficiente. Além disso, a aplicação repetida de `fatorial` dentro da série resulta em recálculos desnecessários. Essas limitações serão discutidas posteriormente na seção de resultados e perspectivas futuras.

Do ponto de vista teórico, o erro cometido ao truncar a série de Taylor em n termos pode ser limitado pelo **resto de Lagrange**. Para a função seno, o resto $R_n(x)$ satisfaz:

$$|R_n(x)| \leq \frac{|x|^{2n+1}}{(2n+1)!} \quad (8)$$

uma vez que as derivadas de $\sin(x)$ são sempre limitadas em módulo por 1. [3] De forma análoga, para a função cosseno, o resto satisfaz:

$$|R_n(x)| \leq \frac{|x|^{2n}}{(2n)!} \quad (9)$$

pelo mesmo argumento, já que as derivadas de $\cos(x)$ também são limitadas em módulo por 1. Essas cotas justificam formalmente o comportamento observado na Tabela 1: como o fatorial no denominador cresce muito mais rapidamente do que a potência de x no numerador, ambos os erros tendem a zero à medida que $n \rightarrow \infty$, garantindo a convergência das aproximações independentemente do valor de x considerado.

4 Resultados

A função `main` não possui uma representação matemática direta como as demais funções apresentadas neste trabalho, pois se trata de uma função impura responsável pela interação do programa com o ambiente externo. [2]

Nesse contexto, a função `main` foi utilizada para fornecer parâmetros às funções desenvolvidas anteriormente e apresentar os resultados obtidos a partir de suas execuções.

```
main :: IO()
main = do
  putStrLn ("Valor do seno: " ++ show (senT pi 10))
  putStrLn ("Valor do cosseno: " ++ show (cosT pi 10))
```

A execução do programa produziu valores aproximados para as funções trigonométricas seno e cosseno utilizando séries de Taylor, tomando $x = \pi$ e $n = 10$ termos. Os resultados obtidos são apresentados na Figura 1.

```
Valor do seno: -5.289182672965142e-10
Valor do cosseno: -1.00000000035290801
```

Figura 1: Saída produzida pelo programa

Os valores retornados por `senT` e `cosT` aproximam-se dos valores reais de $\sin(\pi)$ e $\cos(\pi)$ com uma margem de erro decrescente à medida que n aumenta, o que é consistente com o comportamento de convergência esperado das séries (1) e (3). [3]

Esse comportamento evidencia a relação direta entre o **truncamento** da série – controlado pelo parâmetro n – e a precisão da aproximação obtida: quanto maior o número de termos somados, menor tende a ser o erro em relação ao valor real da função, ao custo de um maior número de avaliações da função `fatorial`. Essa relação entre precisão e custo computacional será retomada na conclusão, como uma das limitações do modelo implementado.

Para ilustrar quantitativamente essa relação, a Tabela 1 apresenta os valores obtidos por `senT` e `cosT` para $x = \pi$ e diferentes quantidades de termos n , comparados aos valores reais $\sin(\pi) = 0$ e $\cos(\pi) = -1$.

Tabela 1: Convergência das aproximações de $\sin(\pi)$ e $\cos(\pi)$				
n	<code>senT</code> (π, n)	erro (sen)	<code>cosT</code> (π, n)	erro (cos)
1	3,1415926536	$3,14 \times 10^0$	1,0000000000	$2,00 \times 10^0$
2	-2,0261201265	$2,03 \times 10^0$	-3,9348022005	$2,93 \times 10^0$
5	0,0069252707	$6,93 \times 10^{-3}$	-0,9760222126	$2,40 \times 10^{-2}$
10	-0,0000000005	$5,29 \times 10^{-10}$	-1,0000000035	$3,53 \times 10^{-9}$

Observa-se que, para valores pequenos de n , a aproximação pode inclusive divergir substancialmente do valor real – como no caso $n = 1$, em que `senT`($\pi, 1$) = π , uma vez que apenas o primeiro termo da série é considerado. À medida que n cresce, no entanto, o erro decresce rapidamente: entre $n = 5$ e $n = 10$, o erro da aproximação do seno cai de ordem 10^{-3} para ordem 10^{-10} , evidenciando a convergência descrita pelas equações (1) e (3). [3]

5 Conclusão

Neste trabalho, foram apresentados os fundamentos matemáticos das séries de Taylor e sua implementação na linguagem Haskell, explorando conceitos característicos do paradigma de programação funcional, como recursão, funções de alta ordem e avaliação preguiçosa (*lazy evaluation*).

A utilização de Haskell permitiu estabelecer uma correspondência direta entre a definição matemática das séries e sua representação computacional, tornando a implementação mais declarativa e próxima da formulação teórica. Além disso, o uso de listas infinitas possibilitou modelar séries matemáticas de forma natural, avaliando apenas os termos necessários para a obtenção das aproximações numéricas. O uso do tipo `Natural` reforçou essa correspondência, restringindo em tempo de compilação o domínio do parâmetro n ao conjunto dos números naturais.

Do ponto de vista matemático, os resultados obtidos foram validados em duas frentes complementares. Empiricamente, a Tabela 1 demonstrou que a utilização de uma quantidade finita de termos é suficiente para produzir boas aproximações das funções seno e cosseno, com o erro caindo de ordem 10^{-2} para ordem 10^{-9} entre $n = 5$ e $n = 10$. Teoricamente, essa convergência não é apenas observada, mas **demonstrada**: as cotas do resto de Lagrange (8) e (9) garantem que o erro de truncamento tende a zero à medida que $n \rightarrow \infty$, o que é consistente com os limites de convergência (2) e (4) estabelecidos na

fundamentação teórica. Dessa forma, o trabalho articula três níveis distintos de evidência, a saber, definição formal, cota analítica de erro e verificação numérica, reforçando a corretude das implementações `senT` e `cosT` para além da simples constatação empírica de que "os valores parecem certos".

Apesar da corretude matemática das implementações, existem limitações relacionadas ao desempenho computacional, principalmente devido ao recálculo repetido da função `fatorial` a cada termo da série, o que introduz uma complexidade evitável. Como perspectivas futuras, o trabalho pode ser expandido por meio da implementação de estratégias de **memoização** para o cálculo do fatorial, bem como da verificação formal das propriedades de convergência das funções `senT` e `cosT` utilizando um assistente de provas, etapa que complementaria a análise do resto de Lagrange aqui apresentada com uma garantia mecanicamente verificada.

6 Referências

Referências

- [1] WIDMAN, Jack. *Aprenda Programação Funcional*. São Paulo: Alta Books, 2025.
- [2] LIPOVAČA, Miran. *Learn You a Haskell for Great Good!*. San Francisco: No Starch Press, 2011.
- [3] BURDEN, Richard L.; FAIRES, J. Douglas. *Análise Numérica*. 10. ed. São Paulo: Cengage Learning, 2016.
- [4] GUIDORIZZI, Hamilton Luiz. *Um Curso de Cálculo*. Vol. 1. Rio de Janeiro: LTC, 2001.
- [5] CANEVARI, Samuel. *Séries de potência*. Notas de aula. São Cristóvão: Universidade Federal de Sergipe (UFS), 2026.
- [6] IEZZI, Gelson. *Fundamentos de Matemática Elementar: Volume 1*. São Paulo: Atual Editora, 2013.